Labo 16 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Requests et limits

Réponse. La **request** est lue par le **scheduler** : il choisit un nœud dont la capacité restante (capacité moins la somme des requests déjà placées) couvre la demande. La **limit** est écrite par le **kubelet** dans le **cgroup** du conteneur. Mémoire au-delà de la limit : le noyau tue le processus (`OOMKilled`, code 137), le kubelet relance (`RESTARTS` + 1, puis `CrashLoopBackOff`). CPU au-delà de la limit : le cgroup **ralentit** le conteneur (*throttling*), rien n'est tué.

Pourquoi. La mémoire ne se « prête » pas : une page allouée l'est jusqu'à libération ; le CPU se partage par tranches de temps.

Nuance. La request CPU sert aussi au partage proportionnel (`cpu.weight`) quand le nœud est saturé ; la request mémoire sert au calcul de l'éviction. Sans limit mémoire, un conteneur peut vider le nœud et provoquer l'éviction d'autres Pods.

Exemple.

```
kubectl describe node | sed -n '/Allocated resources/,/Events/p'   # Requests  Limits  par ressource
kubectl get pod glouton -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'   # OOMKilled
```

Question 2 — QoS et ordre d'éviction

Réponse. `db` est `Guaranteed` (requests = limits), `api` est `Burstable`, `batch` est `BestEffort`. Sous pression mémoire, le kubelet évince d'abord les `BestEffort` (`batch`), puis les `Burstable` qui dépassent leur request (`api`), et les `Guaranteed` en dernier. Si `batch` était un traitement critique, il serait le premier sacrifié, simplement parce que personne n'a déclaré ce qu'il consomme.

Pourquoi. Le kubelet n'a aucune notion de l'importance métier ; il ne connaît que ce que la spec déclare. Un Pod sans `resources` est, par définition, un Pod dont on peut se passer.

Nuance. La QoS ne règle que l'éviction par le **kubelet** ; l'OOM killer du noyau, lui, agit par cgroup quand une limit est dépassée, quelle que soit la classe. Pour un batch important : requests = limits, ou au moins des requests réalistes, et une `PriorityClass` si la plateforme en propose.

Exemple.

```
kubectl get pods -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass
```

Question 3 — `OOMKilled`

Réponse. `kubectl describe pod glouton` montre `Reason: OOMKilled`, `Exit Code: 137` et la limit (`Limits: memory: 32Mi`) ; `kubectl get pod glouton -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'` donne `OOMKilled`. `RESTARTS` augmente parce que `restartPolicy: Always` (défaut) fait relancer le conteneur par le kubelet après chaque mort, avec un délai croissant (`CrashLoopBackOff`). Ici l'application est

conçue pour allouer 100 Mo avec une limit de 32 Mi : les deux sont à revoir — mais dans la vraie vie, c'est presque toujours **l'application** (fuite, tas JVM mal dimensionné) ou une **limit trop juste**, pas la limit « trop basse en soi ».

Pourquoi. 137 = 128 + 9 : le processus a reçu `SIGKILL` du noyau (labo 03, labo 10).

Nuance. Un `OOMKilled` peut aussi frapper un conteneur **sans** limit : si le nœud entier manque de mémoire, le noyau tue le plus gros processus. `describe node` montre alors `MemoryPressure True`.

Exemple.

```
kubectl describe pod glouton | grep -E 'Reason:|Exit Code|Restart Count'
#       Reason:       OOMKilled  /  Exit Code:    137  /  Restart Count:  1
```

Question 4 — Trois Pods `Pending`

Réponse. Le scheduler ne trouve plus de nœud dont la mémoire **réservable** couvre une request de plus ; les Pods restent `Pending` avec `FailedScheduling: 0/1 nodes are available: 1 Insufficient memory`. Corrections possibles : (1) **baisser la request** de l'API si elle est surévaluée (mesurer avec `kubectl top pods`) ; (2) **ajouter un nœud** ou de la mémoire au nœud (`.wslconfig` sur WSL) ; (3) **libérer** le nœud d'autres Pods (ou réduire les réplicas). La plus honnête est (1) **si** la mesure montre que l'API tourne bien en dessous de sa request ; sinon c'est (2) : une request réaliste n'est pas un réglage, c'est un fait.

Pourquoi. Le placement se fait sur les **requests**, pas sur la consommation réelle : un nœud peut être « plein » à 95 % de requests avec 20 % d'utilisation.

Nuance. Baisser la request en dessous de l'usage réel fait passer le Pod en `Burstable` surconsommant : il sera évincé en premier sous pression. Un cluster managé ajoute un nœud automatiquement (*cluster autoscaler*) ; k3s sur WSL, non.

Exemple.

```
kubectl describe pod trop-gourmand | grep FailedScheduling
#  Warning  FailedScheduling  … 0/1 nodes are available: 1 Insufficient memory. …
```

Question 5 — LimitRange

Réponse. (a) Le Pod reçoit `requests: {memory: 32Mi}` et `limits: {memory: 128Mi}` injectés dans sa spec à la création (visibles dans `kubectl get pod -o yaml`). (b) Le Pod est **refusé** à l'`apply` : `maximum memory usage per Container is 1Gi, but limit is 4Gi`. (c) Le Pod garde sa request de `64Mi` et reçoit `limits.memory: 128Mi` par défaut. L'équipe plateforme impose ces objets pour qu'aucun Pod ne tourne sans ressources déclarées (donc en `BestEffort`) et qu'aucun ne puisse réserver un nœud entier.

Pourquoi. Le LimitRange agit comme un **admission controller** : il modifie (défauts) ou rejette (max/min) chaque Pod au moment de sa création.

Nuance. Le LimitRange ne touche pas aux Pods déjà créés ; et si `defaultRequest` dépasse `default`, l'objet est invalide. Un `default` sans `defaultRequest` donne request = limit (donc `Guaranteed`).

Exemple.

```
kubectl run sans-limites --image=docker.io/library/alpine --command -- sleep 3600
kubectl get pod sans-limites -o jsonpath='{.spec.containers[0].resources}'
# {"limits":{"cpu":"250m","memory":"128Mi"},"requests":{"cpu":"50m","memory":"32Mi"}}
```

Question 6 — ResourceQuota dépassée

Réponse. Le budget `requests.memory` restant est `1Gi − 3×64Mi = 832Mi` ; huit Pods de 128Mi en demandent 1024Mi. Le **ReplicaSet** crée les Pods un à un : six passent (768Mi), le septième est refusé. `kubectl get pods` montre **six** Pods du second Deployment, pas huit ; `kubectl describe deployment` dit `6/8` (`ReplicaFailure True`), et le refus se lit dans **`kubectl describe replicaset`** (ou `kubectl get events`) : `exceeded quota: budget, requested: requests.memory=128Mi, used: 896Mi, limited: 1Gi`.

Pourquoi. La quota est vérifiée à la création de chaque **Pod** par l'API server ; le Deployment, lui, est accepté — il ne consomme rien.

Nuance. La limite `pods: 10` jouerait aussi : 3 + 8 = 11 > 10, donc au plus sept Pods de toute façon. Le ReplicaSet réessaie indéfiniment : dès que du budget se libère, les Pods manquants apparaissent.

Exemple.

```
kubectl describe quota budget | sed -n '/Resource/,$p'
# requests.memory  160Mi  1Gi  …
```

Question 7 — `logs --previous`

Réponse. `kubectl logs` affiche le conteneur **en cours** : après un redémarrage, c'est un nouveau processus, dont les logs commencent au démarrage. La cause de la mort est dans la sortie du conteneur **précédent**, que le kubelet conserve pour une seule génération : `--previous`. `lastState` montre `{"terminated": {"reason":"OOMKilled"|"Error"|"Completed","exitCode":137,"finishedAt":…}}` : la raison et le code du dernier conteneur mort.

Pourquoi. Chaque redémarrage crée un nouveau conteneur ; le runtime garde le précédent (et son journal) jusqu'au suivant.

Nuance. Deux redémarrages plus tard, le premier journal est perdu : d'où la collecte centralisée des logs (Loki, Elastic) en production. Et `--previous` échoue sur un Pod qui n'a jamais redémarré.

Exemple.

```
kubectl exec $POD -- sh -c 'kill 1' ; sleep 5
kubectl logs $POD --previous | tail -1     # API arretee proprement.
```

Question 8 — `kubectl debug --target`

Réponse.

```
kubectl debug $POD -it --image=docker.io/library/alpine --target=api --profile=general -- sh
```

`--target=api` place le conteneur éphémère dans le **namespace de processus** du conteneur `api` : `ps` montre le `java -jar`, `/proc/1/root/` donne accès à son système de fichiers, et `localhost:8080` est son port (namespace réseau du Pod). C'est préférable à un shell dans l'image parce que l'image reste minimale et sans outils pour un attaquant (labo 05), et que le shell de diagnostic est choisi **au moment** du besoin, avec les outils qu'il faut (`netshoot` pour le réseau, `alpine` pour un `ps`).

Pourquoi. Les conteneurs éphémères sont un champ `ephemeralContainers` de la spec du Pod, ajouté sans redémarrer le Pod ; ils ne peuvent pas être retirés, mais disparaissent avec lui.

Nuance. Sans `--target`, le conteneur éphémère partage le réseau mais pas les processus. `kubectl debug node/wsl-ubuntu` ouvre de même un shell sur le nœud. Un message `--profile=legacy is deprecated` apparaît si vous omettez `--profile`.

Exemple.

```
kubectl debug $POD -it --image=docker.io/library/alpine --target=api -- sh -c 'ps aux | head
-3; wget -qO- localhost:8080/actuator/health'
#     1 1000      0:01 java -jar /app/api.jar   …   {"status":"UP"}
```

Question 9 — `Forbidden`

Réponse. `Error from server (Forbidden)` : l'API server a authentifié l'appelant puis refusé l'autorisation. `User "system:serviceaccount:lab16:reader"` : le sujet est le ServiceAccount `reader` du namespace `lab16` (préfixe `system:serviceaccount:`). `cannot list resource "secrets" in API group ""` : le verbe `list`, la ressource `secrets`, du groupe *core* (vide). `in the namespace "lab16"` : la portée testée. C'est **le comportement voulu** : le Role ne liste que `pods` et `pods/log`. Pour les ConfigMaps : ajouter une règle `resources: ["configmaps"], verbs: ["get", "list", "watch"]` — et surtout pas `secrets`.

Pourquoi. Le RBAC est **additif et par défaut fermé** : tout ce qui n'est pas explicitement accordé est refusé.

Nuance. `--as` demande le droit `impersonate`, que `cluster-admin` a ; c'est ce qui permet de tester sans créer de kubeconfig. Le ClusterRole `view` livré exclut aussi les Secrets, exprès.

Exemple.

```
kubectl auth can-i get secrets --as=system:serviceaccount:lab16:reader    # no
kubectl auth can-i get pods    --as=system:serviceaccount:lab16:reader    # yes
```

Question 10 — Role vs ClusterRole

Réponse. Role + RoleBinding : des droits sur les ressources **d'un namespace** (l'équipe facturation peut déployer dans `facturation-prod`). **ClusterRole + ClusterRoleBinding** : des droits **sur tout le cluster**, y compris les ressources non-namespacées (nœuds, PV, namespaces) : l'équipe plateforme, un agent de supervision qui lit tous les Pods. **ClusterRole + RoleBinding** : un rôle défini **une fois** au niveau du cluster, mais **accordé dans un seul namespace** : c'est ainsi qu'on donne `edit` sur `facturation-prod` sans le redéfinir partout. Les ClusterRoles `view` / `edit` / `admin` existent justement pour être liés par RoleBinding namespace par namespace.

Pourquoi. Un RoleBinding restreint toujours le rôle référencé à son propre namespace, quel que soit le type du rôle.

Nuance. `admin` via RoleBinding permet de gérer le RBAC du namespace, mais pas de créer des namespaces ni de toucher aux quotas. `cluster-admin` par ClusterRoleBinding est la clé de tout : à réserver aux humains de la plateforme, jamais à un ServiceAccount applicatif.

Exemple.

```
kubectl create rolebinding dev-edit --clusterrole=edit --user=alice -n facturation-prod
kubectl auth can-i create deployments --as=alice -n facturation-prod   # yes
kubectl auth can-i create deployments --as=alice -n default            # no
```

Question 11 — NetworkPolicy

Réponse. Premier `wget` : échec (`wget: can't connect to remote host (10.43.x.x): Connection refused`, ou *timeout*). Second, après le label `app=web` : `{"status":"UP"}`. Dès qu'une NetworkPolicy **sélectionne** un Pod (`podSelector: app=api`), tout trafic entrant non listé est refusé ; avant, en l'absence de politique, tout passait. Sur un cluster dont le CNI n'applique pas les politiques, l'objet est accepté par l'API et **ignoré** : les deux `wget` réussissent, sans aucun message d'erreur.

Pourquoi. La NetworkPolicy est déclarative ; c'est le CNI (kube-router dans k3s) qui la traduit en règles de filtrage sur le nœud, en lisant les labels en continu — d'où l'effet immédiat d'un `kubectl label`.

Nuance. La politique ne regarde que les Pods : le ClusterIP est traduit avant le filtrage, et l'IP source est celle du Pod client. Le refus se manifeste selon le CNI par un `refused` ou un silence (`timeout`) ; testez toujours avec `--timeout`.

Exemple.

```
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health   # échec
kubectl label pod client app=web
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health   #
{"status":"UP"}
```

Question 12 — Politiques pour la stack

Réponse. Deux politiques d'`ingress` : (1) `podSelector: app=db`, `from: podSelector app=api`, port 5432 ; (2) `podSelector: app=api`, `from:` deux sources, `podSelector app=web` **et** Traefik, port 8080. La difficulté : Traefik tourne dans `kube-system`, et un `podSelector` seul ne regarde que le namespace courant ; il faut un `namespaceSelector` (label `kubernetes.io/metadata.name: kube-system`) **combiné** au `podSelector` de Traefik dans le **même** élément de `from` (deux éléments séparés feraient un OU). On commence par une politique *deny-all* (`podSelector: {}` sans `ingress`) parce qu'elle rend le refus **par défaut** : un nouveau Pod n'est joignable que si quelqu'un l'a décidé.

Pourquoi. Sans politique qui le sélectionne, `web` reste joignable par tout le monde — c'est voulu, puisqu'il est servi par l'Ingress ; mais tout autre Pod ajouté demain serait ouvert.

Nuance. Un *deny-all* en `egress` coupe aussi le **DNS** : il faut autoriser `kube-system`/CoreDNS sur 53 UDP/TCP, piège classique. Et les sondes du kubelet ne sont pas concernées : elles viennent du nœud, pas d'un Pod.

Exemple.

```
ingress:
  - from:
      - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }
        podSelector: { matchLabels: { app.kubernetes.io/name: traefik } }
      - podSelector: { matchLabels: { app: web } }
    ports: [ { port: 8080 } ]
```

Question 13 — Jeton volé

Réponse. Avec `default` sans RoleBinding : le jeton est valide (il s'authentifie) mais **n'autorise rien** ; l'attaquant obtient au mieux `kubectl auth can-i --list` et quelques lectures anonymes (`/version`, discovery). Avec `default` lié à `cluster-admin` : il obtient **tout le cluster** : lire tous les Secrets de tous les namespaces, créer un Pod privilégié sur n'importe quel nœud, donc le nœud lui-même. Les deux mesures du labo : un **ServiceAccount dédié sans droit** (et `automountServiceAccountToken: false` puisque l'API n'appelle pas l'API server), et une **NetworkPolicy** d'`egress` qui empêche le Pod de joindre autre chose que ce qu'il doit (l'API server inclus, si rien ne le justifie).

Pourquoi. Le jeton est un fichier lisible par le processus (`/var/run/secrets/kubernetes.io/serviceaccount/token`) ; tout ce que le processus peut faire, l'attaquant qui l'exécute le peut aussi.

Nuance. Les jetons montés sont aujourd'hui **projetés** et à durée limitée (une heure, renouvelés par le kubelet), ce qui limite une exfiltration ; mais un jeton valide une heure avec `cluster-admin` suffit largement. `kubectl get clusterrolebindings -o wide | grep cluster-admin` est l'audit de base.

Exemple.

```
kubectl get clusterrolebindings -o wide | grep -w cluster-admin        # qui est root du cluster ?
kubectl get pod $POD -o jsonpath='{.spec.serviceAccountName}'          # default ?
```